

お絵描き系プログラマー

安田 蒼太

YASUDA

SOTA

福岡情報ITクリエイター専門学校

ゲームクリエイターコース2年

ゲームプログラマー志望

目次

01. プロフィール

02. 最終作品

03. 制作実績

04. 今後の目標



プロフィール

名前

安田 蒼太

出身

長崎

趣味

ゲーム・イラスト制作・アニメ鑑賞・筋トレ・ストレッチ
読書・アイススケート

GitHub

<https://github.com/4zu1Q/MyGame>

スキル

◆ C・C++	2年	◆ PremirePro	2年
◆ Unity・C#	1年	◆ Word/Excel	4年
◆ GitHub	2年	◆ PowerPoint	2年
◆ Photoshop	2年	◆ ClipStudio	4年
◆ Illustrator	1年		



最終作品



作品名 The Legend Of YASUDA

ジャンル 3Dアクションゲーム

開発環境 DxLib ・ C++

制作期間 5ヶ月

制作時期 2024年10月～2025年2月

GitHub <https://github.com/4zu1Q/TheLegendOfYasudaTheMemoryOfFaces>



ゲームについて

ジャンル
3Dアクションゲーム

最終目標は城内にいるボスを倒し、ラスボスを倒すこと

ボスを倒すため、様々な「カオ」を用いて戦う。「カオ」はボスを倒すことで入手できる。

「カオ」を駆使してボスを倒そう

THE LEGEND OF
YASUDA
THE MEMORY OF FACES

ボス部屋に行き



ボスと戦い



ボスを倒す！



通常時では厳しい時は



カオを使用して戦いを換えよう



全てのボスを倒したらゲームクリア



タイトル



タイトル画面でゲームが始まるワクワク感をタイトル画面で感じられるよう右にいるプレイヤーがボタンを押したらアクションを行うよう工夫した。



プレイヤーの種類

プレイヤーの種類をenumとセーブデータのフラグを使うことで、ボスを倒した時に使用できるよう工夫した。

また、プレイヤー別に能力が変わっていて、ボスとの相性を考えて使うよう工夫した。



ノーマルモード



スピードモード



ラストモード



パワーモード



ショットモード



プレイヤーの挙動



プレイヤーの挙動はメンバ関数ポインタを使うことによって前回作っていたキャラクターの状態遷移よりも簡単に行えるよう工夫した。

メンバ関数ポインタを使用していない

```
//ボタンを押したら攻撃アニメーションを再生する
if (!m_isAttack && !m_isSkill && !m_isDown && !m_isCommand)
{
    //攻撃
    if (Pad::IsPress(PAD_INPUT_3) && !m_isStamina)
    {
        m_isAttack = true;
        ChangeAnim(kAttackAnimIndex);
        m_isMove = true;
        PlaySoundMem(m_axeAttackSeH, DX_PLAYTYPE_BACK, true); //アタック
    }
    else
    {
        if (m_isMove)
        {
            Move();
        }
    }

    //スキル攻撃
    if (Pad::IsPress(PAD_INPUT_4) && m_stamina >= 100.0f && !m_isStamina)
    {
        m_isSkill = true;
        ChangeAnim(kSkillAnimIndex);
        m_isMove = false;
        PlaySoundMem(m_axeSkillSeH, DX_PLAYTYPE_BACK, true); //スキル
        m_stamina -= 100.0f;
    }
    else
    {
        if (!m_isMove)
        {
            Move();
        }
    }
}
else
```

Update関数内がとても見やすい

メンバ関数ポインタを使用

```
void Player::Update(std::shared_ptr<MyLib::Physics> physics,
{
    //アップデート
    (this->*m_updateFunc)();

    //RTのインプットを取得
    GetJoypadDirectInputState(DX_INPUT_PAD1, &m_input);

    //アニメーションの更新処理
    m_pAnim->UpdateAnim();
    m_bossAttackKind = bossAttackKind;

    //ダメージの更新処理
    DamageUpdate();

    //プレイヤーの当たり判定座標の更新
    CollisionPosUpdate();

    //ボスの座標を代入
    m_isLockOn = isLockOn;
    m_bossPos = bossPos;
    m_bossToPlayerVec = VSub(m_pos, m_bossPos);

    //プレイヤーモデルの回転処理
    PlayerSetPosAndRotation(m_pos, m_angle);

    //プレイヤーのステータス更新
    StatusUpdate();
}
```


ボスのAI

ボスのAIはプレイヤーとの線の距離とメンバ関数ポインタを使うことでプレイヤーがどこにいても戦うよう工夫した。

距離が遠すぎると走ってくる



距離が近くなると歩いてくる



攻撃できる距離になったら止まって攻撃する



地形との当たり判定

プレイヤーと地形の当たり判定はポリゴンで処理している。
当たり判定はフィールドのポリゴンの法線の向きによって壁と床に分類して処理を行っている。

青：床 赤：壁

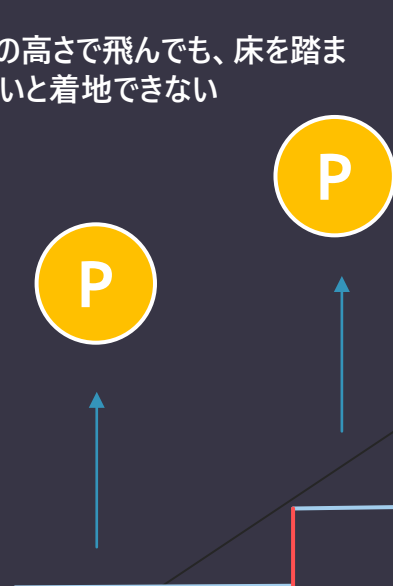


壁の判定

床の判定

どの高さで飛んでも、床を踏まないと着地できない

青：床 赤：壁

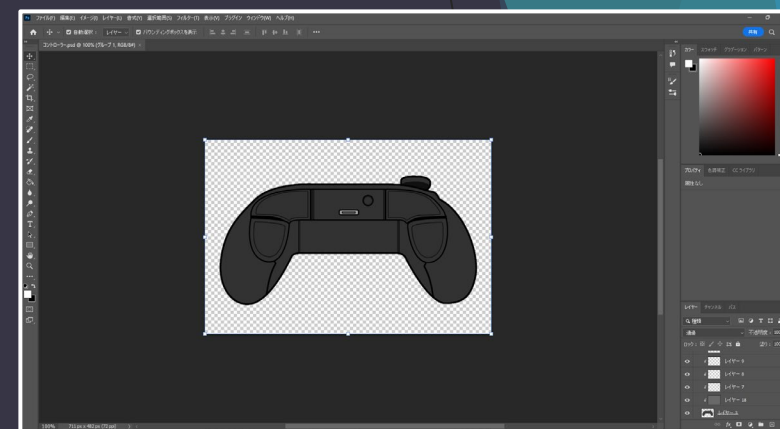


UI①



UIの見やすさを意識し作成した。元の操作説明のUIはどこのボタンがどの役割をもっているのかとても見づらい配置になっていたのを現在の見やすい配置になるよう工夫した。

フォトショでコントローラーを一から作成



前のUI



とても見やすい！！

今のUI

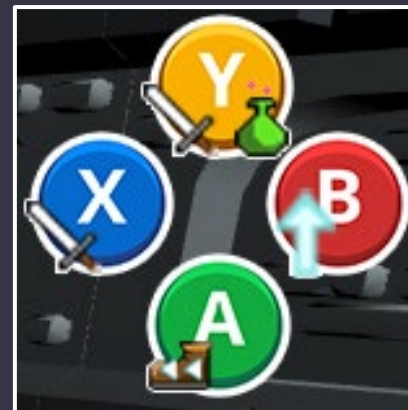


UI②

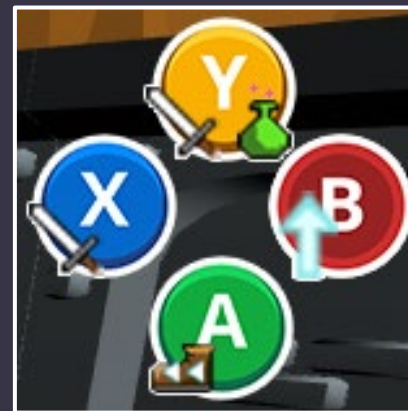
ジャンプやダッシュボタンを押すと、ゲーム画面内でそのボタンが押されていることが視認できるよう作成した。

ゲーム内のアイテムを使用している間はアイテムを選べるUIの色を変えることで今アイテムを使用しているとわかりやすく工夫した。

ジャンプボタンを押す前



ジャンプボタンを押した後



押したボタンが動く

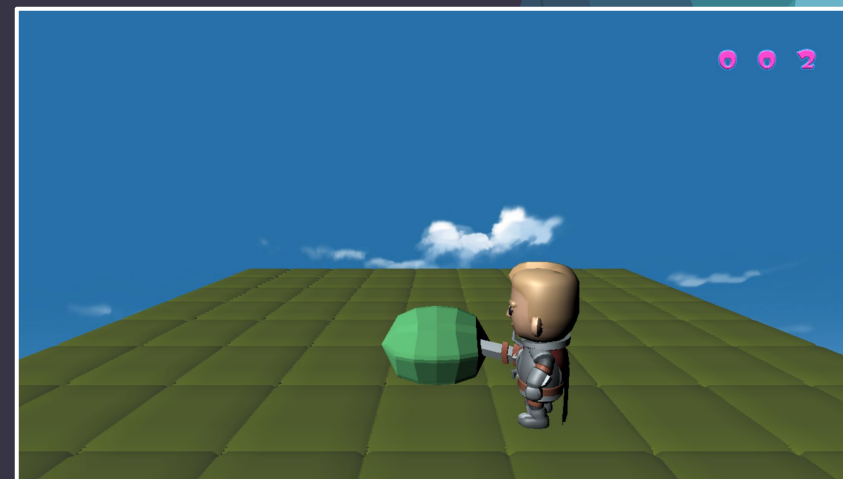
制作実績 個人製作①

作品名	小人の旅路 (個人制作)
ジャンル	3Dアクション
GitHub	https://github.com/4zu1Q/TheDwarf-sJourney
開発環境	DxLib ・ C++
制作期間	2ヶ月
制作時期	2023年11月～2024年1月
学習内容	プレイヤーの挙動、敵の挙動



制作実績 個人製作②

作品名	連打でスイカ！
ジャンル	ミニゲーム
GitHub	https://github.com/4zu1Q/Streak-WatermelonSlap
開発環境	DxLib ・ C++
制作期間	1ヶ月
制作時期	2024年5月～2024年6月
学習内容	3Dモデルの描画、アニメーション



制作実績 個人製作③

作品名	ActionToAvoid
ジャンル	2Dアクション
GitHub	https://github.com/4zu1Q/ActionToAvoid
開発環境	DxLib ・ C++
制作期間	3ヶ月
制作時期	2023年11月～2024年1月
学習内容	プレイヤーの挙動、マップチップ 当たり判定



制作実績 個人製作④

作品名	ジャンプキャット
ジャンル	2Dジャンプアクション
GitHub	https://github.com/4zu1Q/JumpCat
開発環境	Unity ・ C#
制作期間	1ヶ月
制作時期	2023年8月～2023年9月
学習内容	ランダムドロップ、プレイヤー挙動



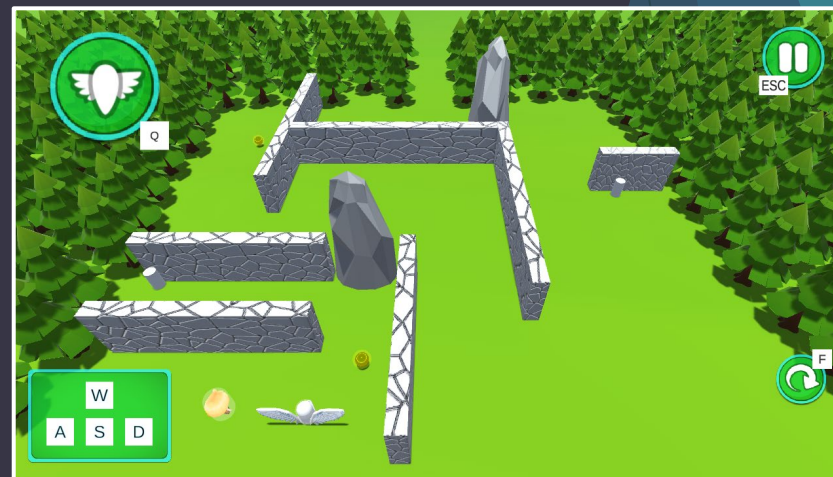
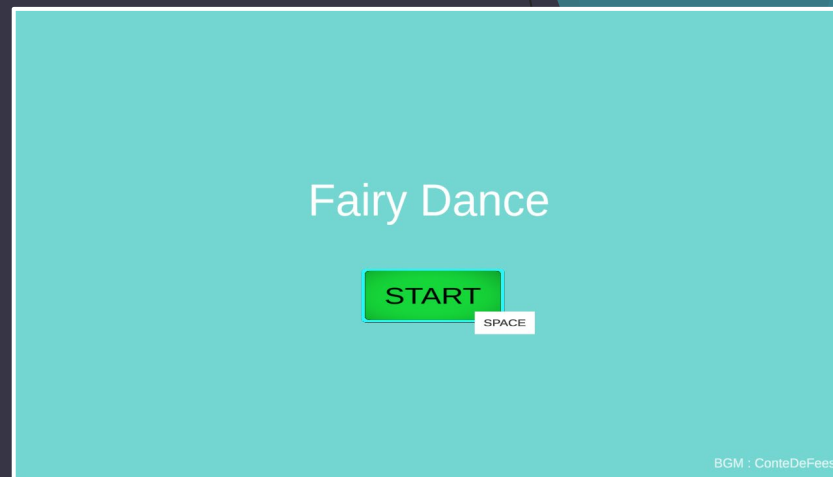
制作実績 チーム製作①

作品名	SwordTrial
ジャンル	3Dアクション
GitHub	https://github.com/4zu1Q/SwordTrials
開発環境	Unity ・ C#
制作期間	3ヶ月
制作時期	2023年11月～2024年1月
担当箇所	プレイヤーの挙動、進捗管理、仕様制作
学習内容	進捗管理、仕様制作



制作実績 チーム製作②

作品名	FearyDance
ジャンル	3Dアクションパズル
GitHub	https://github.com/4zu1Q/FairyDance
開発環境	Unity ・ C#
制作期間	3ヶ月
制作時期	2023年11月～2024年1月
担当箇所	ギミック実装、ステージ制作
学習内容	共同制作、ギミック処理



今後の目標

- プレイヤーの挙動や攻撃判定などの処理を見直し、もっと気持ち良く操作し攻撃できる挙動の作成。
- 自分がチャレンジできていない技術に触れ、プログラマーとしての経験を積む。
- プログラマーだけの視点を立つだけではなく、デザイナーやプランナーの視点に立てるよう考える。